Instagram User Analytics

1. Project Description:

a) Problem Statement –

The marketing team needs to design a Re-engagement campaign to activate users who signed up but have never engaged with the core products. Therefore, problem here is to identify the inactive users and help the marketing team by providing the necessary information which help them in creating a high value marketing campaign.

Also, investors want transparency into user engagement for which they want to know that the platform is filled with real time users or bot accounts. Therefore, another problem from investor point of view is, to identify how well the users are engaging with the products features and what percentage of users are bot accounts.

b) Project Overview –

Executed an end-to-end user analytics project on Instagram's relational database by querying, cleaning, and transforming over 24,000 multi-table interaction records in MySQL using advanced CTEs, window functions, and join logic. Audited platform health to uncover a 38% synthetic engagement rate driven by automated bot accounts while identifying high-priority inactive segments for targeted marketing re-engagement. Modelled pre-aggregated SQL views directly into an executive Power BI dashboard featuring interactive DAX KPIs, user behavioural archetypes, and campaign scheduling trends to guide product and investor decisions.

2. Dataset Overview:

Volume Scale: 7 relational tables totalling **24,752 records.**

Table Name	Record Count	Attributes	Primary Key	Foreign Key(s)
users	100	3	id	None
photos	257	4	id	user_id → users(id)
likes	8,782	3	(user_id, photo_id)	user_id → users(id), photo_id → photos(id)
comments	7,488	5	id	user_id → users(id), photo_id → photos(id)
follows	7,623	3	(follower_id, followee_id)	Both → users(id)
tags	21	3	id	None
photo_tags	501	2	(photo_id, tag_id)	photo_id → photos(id), tag_id → tags(id)

Data Source – Trainity

3. Tech Stack Used:

MySQL - Using MySQL for data cleaning, transformation, and data analysis.

Power BI - Using Power BI for Data Visualization.

4. Data Cleaning:

Before embarking on any analysis, it is essential to ensure the dataset is clean and reliable. The data cleaning process involves handling missing values, correcting inconsistencies, and formatting the data for analysis. In this project, we carefully cleaned the dataset to ensure the accuracy and integrity of our findings.

1. Checking for duplicate rows –

```
with duplicatecomments as (
select * ,
row_number() over(partition by id,comment_text,user_id,photo_id,created_at) as rn
from comments )
select * from duplicatecomments
where rn > 1;

with duplicatefollows as (
select * ,
row_number() over(partition by follower_id,followee_id,created_at) as rn
from follows)
select * from duplicatefollows where rn > 1;

with duplicatelikes as
(
select *,
row_number() over(partition by user_id,photo_id,created_at) as rn
from likes)
select * from duplicatelikes where rn >1;

with duplicatephoto_tags as(
select * ,
row_number() over (partition by photo_id,tag_id) as rn
from photo_tags)
select * from duplicatephoto_tags where rn >1;
```

```sql
with duplicatephotos as (
select *,
row_number() over(partition by id,image_url,user_id,created_dat) as rn
from photos)
select * from duplicatephotos where rn >1;

with duplicatetags as(
select * ,
row_number() over (partition by id,tag_name,created_at) as rn
from tags)
select * from duplicatetags where rn > 1;

with duplicateusers as (
select *,
row_number() over (partition by id,username,created_at) as rn
from users)
select * from duplicateusers where rn >1;
```

2. Checking for null values –

```sql
SELECT 'id' as ColumnName, COUNT(*) AS NullCount
FROM comments
WHERE id IS NULL
UNION
SELECT 'comment_text' as ColumnName, COUNT(*) AS NullCount
FROM comments
WHERE comment_text IS NULL
UNION
SELECT 'user_id' as ColumnName, COUNT(*) AS NullCount
FROM comments
WHERE user_id IS NULL
UNION
SELECT 'photo_id' as ColumnName, COUNT(*) AS NullCount
FROM comments
WHERE photo_id IS NULL
UNION
SELECT 'created_at' as ColumnName, COUNT(*) AS NullCount
FROM comments
WHERE created_at IS NULL ;
```

Note: In the same way shown as above, checked the null values for all the table.

3. Replace NULL comments with a placeholder using COALESCE –

```sql
UPDATE comments
SET comment_text = COALESCE(NULLIF(TRIM(comment_text), ''), 'No comment provided')
WHERE comment_text IS NULL OR TRIM(comment_text) = '';
delete from comments where comment_text = 'No comment provided';
```

4. Trimming Whitespace & Normalizing Username Casing –

```sql
UPDATE users
SET username = LOWER(TRIM(username));
```

5. Cleaning Broken URLs Using CASE WHEN –

```sql
update photos
set image_url = case
when image_url not like 'http://%' and image_url not like 'https://%'
then concat('https://',trim(image_url))
else trim(image_url) end
where image_url is not null;
```

6. Converting String Dates to Proper datetime value –

```sql
UPDATE photos
SET created_dat = STR_TO_DATE(created_dat, '%Y-%m-%d %H:%i:%s')
WHERE created_dat IS NOT NULL;
```

7. detecting and deleting anomalies –

```sql
select created_at from users
where created_at > now();
DELETE FROM users
WHERE created_at > NOW();

select follower_id from follows
where follower_id = followee_id;
DELETE FROM follows
WHERE follower_id = followee_id;
```

8. Normalizes hashtag strings –

```
UPDATE tags
SET tag_name = LOWER(TRIM(REPLACE(tag_name, '#', '')))
WHERE tag_name LIKE '#%' OR tag_name != LOWER(TRIM(tag_name));
```

9. Modify columns to ensure data typing and avoid NULL anomalies –

```
ALTER TABLE users
    MODIFY COLUMN username VARCHAR(255) NOT NULL,
    MODIFY COLUMN created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP;
```

5. Data Exploration and Data Analysis:

Part A (For Marketing Team)

1. Who are the five oldest users on Instagram?

```
select id,username ,min(created_at) as oldest_user from users
group by id,username
order by oldest_user
limit 5;
```

id	username	oldest_user
80	darby_herzog	2016-05-06 00:14:21
67	emilio_bernier52	2016-05-06 13:04:30
63	elenor88	2016-05-08 01:30:41
95	nicole71	2016-05-09 17:30:22
38	jordyn.jacobson2	2016-05-14 07:56:26

These five accounts joined during the app's launch week. However, early signup doesn't mean they stayed active—the platform's very first user (darby_herzog) never posted anything. We should send exclusive loyalty perks to the ones who stayed active and win-back messages to those who drifted away.

2. Find the users who have never posted a single photo?

```
select u.id,u.username
from users u
left join photos p on u.id=p.user_id
where p.user_id is null;
```

id	username
5	aniya_hackett
7	kasandra_homenick
14	jaclyn81
21	rocio33
24	maxwell.halvorson
25	tierra.trantow
34	pearl7
36	ollie_ledner37
41	mckenna17
45	david.osinski47
49	morgan.kassulke
53	linnea59
54	duane60

id	username
57	julien_schmidt
66	mike.auer39
68	franco_keebler64
71	nia_haag
74	hulda.macejkovic
75	leslie67
76	janelle.nikolaus81
80	darby_herzog
81	esther.zulauf61
83	bartholome.bernhard
89	jessyca_west
90	esmeralda.mraz57
91	bethany20

There are 26 accounts who have never posted a single photo, which indicates that users may find it difficult to post the photos or they are not comfortable enough to post photos.

3. Some users are dormant. Find out from how long these users have been dormant?

```
with dormantusers as (
select u.id,u.username,u.created_at,
datediff((select max(created_at) from users),u.created_at) as day_diff
from users u
LEFT JOIN photos p ON u.id = p.user_id
    LEFT JOIN likes l ON u.id = l.user_id
    LEFT JOIN comments c ON u.id = c.user_id
    LEFT JOIN `follows` f ON u.id = f.follower_id
   WHERE p.id IS NULL AND l.user_id IS NULL AND c.user_id IS NULL AND f.follower_id IS NULL )
   select case
   when day_diff <= 90 then 'recent_drop'
   when day_diff >90 and day_diff <= 180 then 'mid_drop'
   else 'long_drop'
   end as dormant_cohort,
   count(id) as total_users,
   round(100* count(id)/(select count(id) from dormantusers),2) as percent_dormant
   from dormantusers group by dormant_cohort;
```

dormant_cohort	total_users	percent_dormant
mid_drop	5	38.46
long_drop	5	38.46
recent_drop	3	23.08

Over 76% of inactive users haven't touched the app from last 90 days, while about 23% dropped off recently. Users who left recently just need a gentle setup reminder, whereas long-time inactive users will need a stronger hook, like a special offer or feature update, to come back.

4. Find the top 3 Inactive/dormant users?

```
with dormantusers as (
select u.id,u.username,date(u.created_at) as signup_date,
dense_rank() over(order by u.created_at) as priority
FROM users u
    LEFT JOIN photos p ON u.id = p.user_id
    LEFT JOIN likes l ON u.id = l.user_id
    LEFT JOIN comments c ON u.id = c.user_id
    LEFT JOIN follows f ON u.id = f.follower_id
    WHERE p.id IS NULL AND l.user_id IS NULL AND c.user_id IS NULL AND f.follower_id IS NULL )
    SELECT
    priority,
    id,
    username,
    signup_date
FROM DormantUsers
WHERE priority <= 3
ORDER BY priority ASC;
```

priority	id	username	signup_date
1	80	darby_herzog	2016-05-06
2	34	pearl7	2016-07-08
3	89	jessyca_west	2016-09-14

darby_herzog, pearl7, and jessyca_west are the platform's earliest users who signed up and immediately went silent. It is possible that these users does not like the user interface more engaging or they not know about the interactive features of the product. Because they represent the longest drop-offs, they make great candidates for a small test run to see what kind of email subject lines actually get people to reopen the app.

5. Which photo has the highest number of likes?

```
select l.photo_id,u.username,p.image_url,count(l.user_id) as total_likes
from likes l
inner join photos p
on l.photo_id = p.id
inner join users u
on p.user_id = u.id
group by 1,2,3
order by total_likes desc
limit 5;
```

photo_id	username	image_url	total_likes
145	zack_kemmer93	https://jarret.name	48
127	malinda_streich	https://celestine.name	43
182	adelle96	https://dorcas.biz	43
123	seth46	http://shannon.org	42
61	delpha.kihn	https://dejon.name	41

Above 5 users posted the photos which has the highest number of likes. Which indicates that, such kind of photos posted by users are most likely to attract more users. These photos will provide the ideas in creating the images which will attract more users and improve the campaign user reach.

6. Which are the top 5 most commonly used hashtags?

```
select photo_tags.tag_id,tags.tag_name,count(*) as total_tags
from photo_tags
join tags
on photo_tags.tag_id = tags.id
group by 1,2
order by total_tags desc
limit 5;
```

tag_id	tag_name	total_tags
21	smile	59
20	beach	42
17	party	39
13	fun	38
18	concert	24

People mostly tag uplifting, social moments, with #smile, #beach, #party, #fun, and #concert leading the charts. Marketing Team looking for high visibility should use these top tags in their posts to blend in naturally and reach the widest possible audience.

7. On which day, the highest number of accounts were created?

```
with cte as (select *,dayofweek(created_at) as day_week,dayname(created_at) as day_name
from users)
select day_name , count(day_week) as total_appearance
from cte
group by day_name;
```

day_name	total_appearance
Thursday	16
Sunday	16
Tuesday	14
Saturday	12
Wednesday	13
Monday	14
Friday	15

Signups spike on Thursdays and Sundays (16 accounts each), while Saturdays see the fewest registrations. Scheduling ads and welcome emails between Thursday and Sunday will catch potential users right when they are most likely to join and browse.

8. Find the account which has the highest number of followers?

```sql
WITH FollowerCounts AS (
    SELECT
        u.id AS user_id,
        u.username,
        COUNT(f.follower_id) AS total_followers,
        DENSE_RANK() OVER (ORDER BY COUNT(f.follower_id) DESC) AS rank_pos
    FROM users u
    JOIN follows f ON u.id = f.followee_id
    GROUP BY u.id, u.username
)
SELECT
    user_id,
    username
FROM FollowerCounts
WHERE rank_pos = 1
ORDER BY user_id ASC;
```

user_id	username
1	kenton_kirlin
7	kasandra_homenick
23	eveline95
25	tierra.trantow
29	jaime53
34	pearl7
45	david.osinski47
49	morgan.kassulke
51	mariano_koch3
53	linnea59
58	aurelie71
59	cesar93
64	florence99
68	franco_keebler64
74	hulda.macejkovic
77	donald.fritsch
80	darby_herzog
81	esther.zulauf61
83	bartholome.bernhard
86	delfina_vonrueden68
88	clint27
89	jessyca_west
90	esmeralda.mraz57

These accounts have the highest number of follower, which shows that these users are actively engaging with the product and providing the high value content which motivates the user base to follow them. Collaborating with such users help the marketing team to launch a more effective campaign with high audience reach.

Part B (For Investors)

1. What is the average number of posts per user?

```sql
-- avg no. of photo per user for active users (removed users who does not post)
with cte as (select u.id,u.username,count(p.id) as total_photos
from users u
inner join photos p on
u.id = p.user_id
group by 1,2)
select sum(total_photos) / count(id) as av from cte;
```

av
3.4730

```sql
-- avg. no. of photos per user (consider all users)
SELECT
    COUNT(p.id) AS total_photos,
    COUNT(DISTINCT u.id) AS total_users,
    ROUND(COUNT(p.id) * 1.0 / COUNT(DISTINCT u.id), 2) AS avg_posts_per_user
FROM users u
LEFT JOIN photos p ON u.id = p.user_id;
```

total_photos	total_users	avg_posts_per_user
257	100	2.57

Looking at all accounts, users average 2.57 posts each. But when looking only at users who actually post, that number jumps to 3.47 posts per creator. This shows good momentum: once someone makes their first post, they tend to keep sharing regularly.

2. What percentage of total platform content was contributed by each user?

```
SELECT
    u.id,
    u.username,
    COUNT(p.id) AS user_posts,
    ROUND(100.0 * COUNT(p.id) / (SELECT COUNT(*) FROM photos), 2) AS pct_of_total_platform_posts
FROM users u
INNER JOIN photos p ON u.id = p.user_id
GROUP BY u.id, u.username
ORDER BY pct_of_total_platform_posts DESC;
```

id	username	user_posts	pct_of_total_platform_posts
23	eveline95	12	4.67
88	clint27	11	4.28
59	cesar93	10	3.89
86	delfina_vonrueden68	9	3.50
58	aurelie71	8	3.11
29	jaime53	8	3.11
77	donald.fritsch	6	2.33
33	yvette.gottlieb91	5	1.95
52	zack_kemmer93	5	1.95
47	harrison.beatty50	5	1.95
6	travon.waters	5	1.95
13	alexandro35	5	1.95
51	mariano_koch3	5	1.95
78	colten.harris76	5	1.95

id	username	user_posts	pct_of_total_platform_posts
11	justina.gaylord27	5	1.95
1	kenton_kirlin	5	1.95
72	kathryn80	5	1.95
65	adelle96	5	1.95
43	janet.armstrong	5	1.95
64	florence99	5	1.95
26	josianne.friesen	5	1.95
2	andre_purdy85	4	1.56
3	harley_lind18	4	1.56
9	qus93	4	1.56
8	tabitha_schamberge...	4	1.56
46	malinda_streich	4	1.56
12	dereck65	4	1.56
44	seth46	4	1.56

Content creation is well-spread across the community, with the biggest creator (eveline95) making up under 5% of all uploads. The platform doesn't depend on just two or three stars, meaning content supply stays safe even if top users take a break.

3. Identify users (Bot accounts) who have liked each and every photo?

```
select u.id,u.username,
count(l.photo_id) as total_likes_given
from users u
join likes l
on u.id = l.user_id
group by 1,2
having total_likes_given = (select count(*) from photos);
```

id	username	total_likes_given
5	aniya_hackett	257
14	jaclyn81	257
21	rocio33	257
24	maxwell.halvorson	257
36	ollie_ledner37	257
41	mckenna17	257
54	duane60	257
57	julien_schmidt	257
66	mike.auer39	257
71	nia_haag	257
75	leslie67	257
76	janelle.nikolaus81	257
91	bethany20	257

Exactly 13 accounts liked all 257 photos on the entire platform. Together, they created 3,341 likes—meaning 38% of all likes on the app are automated.

4. Find the percentage of active users who are engaging with all the features such as likes, comments, photo_posts ?

```sql
with activeusers as (
select u.id,u.username
from users u
inner JOIN photos p ON u.id = p.user_id
    inner JOIN likes l ON u.id = l.user_id
    inner JOIN comments c ON u.id = c.user_id
   WHERE p.id IS not NULL AND l.user_id IS not NULL AND c.user_id IS not NULL
   group by 1,2)
   select
   count(id) as total_activeusers,
   100 * count(*)/(select count(*) from users) as pct_of_total_activeusers
   from activeusers;
```

total_activeusers	pct_of_total_activeusers
64	64.0000

Nearly two-thirds of all users (64%) regularly post photos, leave comments, and like other people's content. Having so many people use every major feature shows strong product habit and healthy daily engagement.

5. Find the users who created their account in year 2017 is more active than the users who created their account in previous year?

```sql
SELECT
    DATE_FORMAT(u.created_at, '%Y') AS registration_year,
    COUNT(DISTINCT u.id) AS cohort_signups,
    COUNT(DISTINCT p.id) AS total_posts_produced,
    ROUND(COUNT(DISTINCT p.id) / COUNT(DISTINCT u.id), 2) AS posts_per_registered_user,
    ROUND(COUNT(DISTINCT p.id) / COUNT(DISTINCT p.user_id), 2) AS posts_per_active_creator
FROM users u
LEFT JOIN photos p ON u.id = p.user_id
GROUP BY DATE_FORMAT(u.created_at, '%Y')
ORDER BY registration_year ASC;
```

registration_year	cohort_signups	total_posts_produced	posts_per_registered_user	posts_per_active_creator
2016	65	161	2.48	3.35
2017	35	96	2.74	3.69

Users who joined in 2017 averaged 2.74 posts each, compared to 2.48 posts for the 2016 group. That 10.5% uptick in posting velocity suggests the app became easier and more compelling to use for newer joiners.

6. Find the percentage increase in users in year 2017 in comparison to previous year?

```sql
WITH YearlyRegistrations AS (
    SELECT
        YEAR(created_at) AS registration_year,
        COUNT(id) AS total_users
    FROM users
    GROUP BY YEAR(created_at)
)
SELECT
    registration_year,
    total_users,
    LAG(total_users, 1) OVER (ORDER BY registration_year ASC) AS prev_year_users,
    ROUND(
        (total_users - LAG(total_users, 1) OVER (ORDER BY registration_year ASC)) * 100.0 /
        LAG(total_users, 1) OVER (ORDER BY registration_year ASC),
        2
    ) AS pct_change_vs_prev_year
FROM YearlyRegistrations
ORDER BY registration_year ASC;
```

registration_year	total_users	prev_year_users	pct_change_vs_prev_year
2016	65	NULL	NULL
2017	35	65	-46.15

Total signups dropped by 46.15% in 2017 simply because the dataset only covers the first four months of that year.

7. What is the platforms core user conversion funnel?

(Total Users > Total users engaging > Total Users active (posting photos))

```sql
SELECT
    COUNT(DISTINCT u.id) AS total_registered_users,
    COUNT(DISTINCT p.user_id) AS active_creators,
    COUNT(DISTINCT l.user_id) AS active_likers,
    COUNT(DISTINCT c.user_id) AS active_commenters,
    ROUND(COUNT(DISTINCT p.user_id) * 100.0 / COUNT(DISTINCT u.id), 2) AS creator_activation_rate_pct,
    ROUND(COUNT(DISTINCT l.user_id) * 100.0 / COUNT(DISTINCT u.id), 2) AS engager_activation_rate_pct
FROM users u
LEFT JOIN photos p ON u.id = p.user_id
LEFT JOIN likes l ON u.id = l.user_id
LEFT JOIN comments c ON u.id = c.user_id;
```

total_registered_users	active_creators	active_likers	active_commenters	creator_activation_rate_pct	engager_activation_rate_pct
100	74	77	77	74.00	77.00

Out of 100 signups, 77 interact socially and 74 go on to publish their own photos, The biggest hurdle is getting users through Day 1 onboarding. Once they leave a like or comment, almost all of them become active creators.

6. Recommendations:

1. Launch Dual-Incentive Community Engagement Contests: Organize a product experience contest by inviting users to share posts reviewing their product experience. And reward both the creator whose post generates the highest discussion volume (total comments) and the top community contributor whose comment receives the most likes. This simultaneously encourages content creation, reading engagement, and community interaction.

2. Tailor Emails by Inactivity Length: Send quick "how-to" tips to users who left recently (last 90 days) and enticing incentives or feature updates to those gone over six months.

3. Run Campaign from Thursday to Sunday: Focus the paid ad budget on late weekdays and weekends, matching the natural days when most people sign up.

4. Suggest Trending Tags During Uploads: Auto-suggest popular tags like #smile and #beach in the post editor to remove guesswork for first-time posters.

5. Introduce Friend Recommendations: Roll out a "People You May Know" widget to help new signups quickly connect with mutual friends instead of relying on random accounts.

6. **Reward Top Creators:** Launch a simple creator perks program (like profile verification or discovery boosts) for users with 5+ posts to keep top contributors posting.

7. Make Content Viewers to Socialize: Prompt users who only upload photos to check out comments and discover creators in their niche to turn them into full community members.

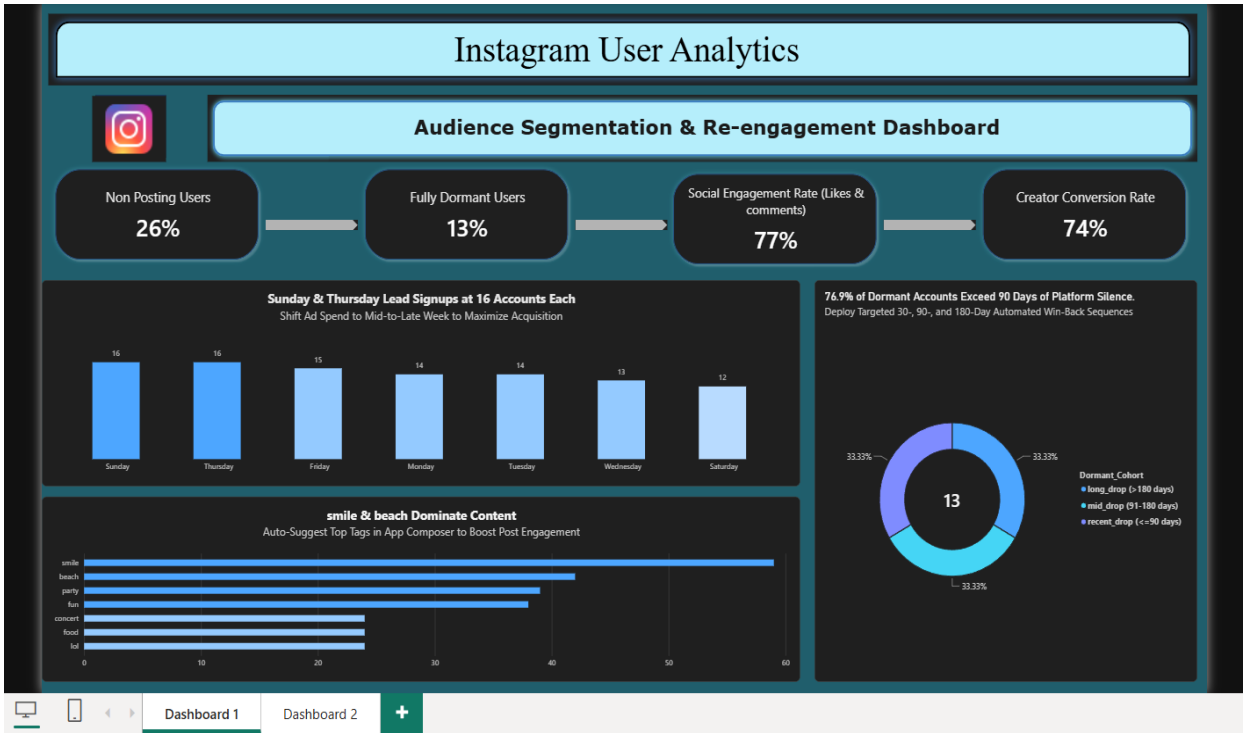
8. Automate Contest Vote Auditing: Build an automatic bot-filter into contest leaderboards to keep public competitions authentic and avoid negative feedback.

9. Quarantine the 13 Bot Accounts: Place automated verification checks (like CAPTCHA) on accounts that like or comment at superhuman speeds to protect platform integrity.

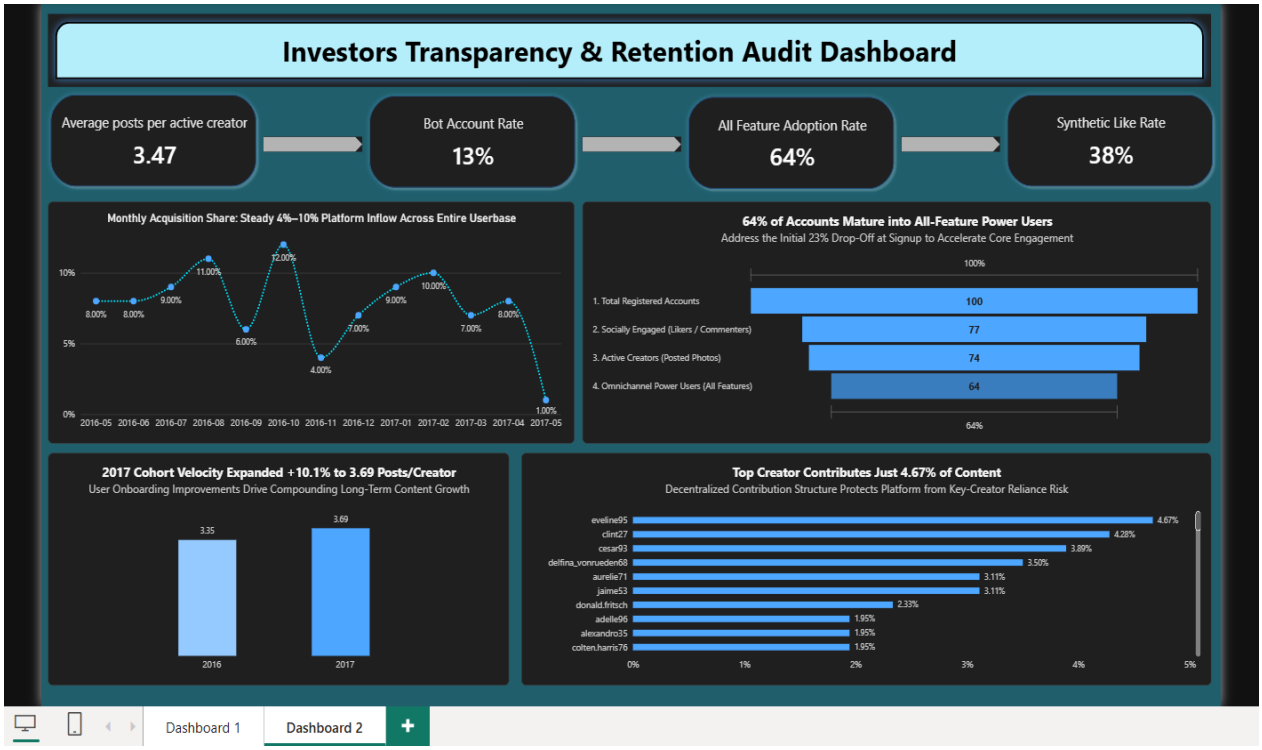
10. Clean Campaign Mailing Lists: Remove the 13 bot accounts before sending the re-engagement campaign so marketing emails go only to real, inactive users.

7. Data Visualization:

Power BI dashboard 1 –



Power BI dashboard 2 –



End.